

Computing the Restricted Algebraic Immunity, and Application to Weightwise Perfectly Balanced Functions.

Luca Bonamino^[0009–0009–3883–6958]¹, Pierrick Méaux^[0000–0001–5733–4341]²

¹ DCS – Department of Computer Science

² SnT – Interdisciplinary Centre for Security, Reliability and Trust
University of Luxembourg, Luxembourg
luca.bonamino.001@uni.lu
pierrick.meaux@uni.lu

Abstract. Algebraic Immunity (AI) is a fundamental property in the security analysis of stream ciphers and Boolean functions, measuring the resistance of a function against algebraic attacks. In this work, we focus on the computation of AI and, more specifically, on its generalization to restricted algebraic immunity, which considers annihilators constrained to specific subsets of $\mathbb{F}_2^n$. While the computation of AI has been studied using methods based on Reed-Muller codes and iterative rank-based algorithms, these approaches have not been formally adapted to the restricted setting. We address this gap by establishing the theoretical foundations required for the adaptation of these techniques and providing explicit algorithms for computing restricted algebraic immunity.

To assess the efficiency of our algorithms, we conduct practical experiments comparing the computational cost of the Reed-Muller and iterative approaches in terms of time and memory. As a case study, we analyze the algebraic immunity restricted to the slices of $\mathbb{F}_2^n$, *i.e.* the sets of binary vectors of fixed Hamming weight, denoted AI_k . The slices are of particular interest in areas of cryptography, such as side-channel analysis and stream cipher design, where the input weight may be partially predictable. We further investigate restricted algebraic immunity for Weightwise (Almost) Perfectly Balanced (W(A)PB) functions, which have been extensively studied since 2017. Our results include an empirical analysis of AI_k distributions for WPB functions with 4, 8, and 16 variables, as well as an evaluation of AI_k for various known function families.

Keywords: Boolean functions, Algebraic immunity, Restricted algebraic immunity, Weightwise (almost) perfectly balanced functions.

1 Introduction

Since the introduction of algebraic attacks on filtered LFSRs [15] in 2003, Algebraic Immunity (AI) has become a crucial property to consider in the security of stream ciphers. As defined in [50], algebraic immunity refers to the minimum degree of a nontrivial annihilator of a Boolean function f or its complement $1 + f$. In the context of filtered LFSRs, AI directly corresponds to the degree of the algebraic system an adversary can derive from the keystream, since only affine mappings occur before the application of the nonlinear filter f . Consequently, AI plays a key role in settings where affine updates are followed by a filtering function, as demonstrated in [1] for Goldreich’s pseudorandom generator [28] and for (improved) filtered permutators [46,47]. Studying algebraic immunity (*e.g.* [8,20,43]) or constructing Boolean functions that optimize this parameter (*e.g.* [4,6,12,16,17,33,35,36,37,39,40,53,55,56,57,58,59,61]) has been an important line of research in the area of Boolean functions used in cryptography since the 2000s. With the recent resurgence of filtered LFSRs [10,11]—now operating on Boolean functions in over 100 variables—or other stream ciphers over $\mathbb{Z}_{2^t}$ that use filtering functions in many variables [13,31] for efficient hybrid homomorphic encryption [32], determining the algebraic immunity (or its variants) has become increasingly relevant for assessing the practical security of these schemes. Following this trend, algebraic immunity has been of the primary criteria studied for the newly proposed functions in [9,44,48]. Concerning recent attacks on AI variants, the attack on Elisabeth-4 in [22] exploits the existence of low-degree annihilators

whose monomials are restricted to a specific subspace. Similarly, the attack framework of [49] investigates AI variants that depend on the updating process of the stream cipher.

We first recall methods for the computation of this parameter, specifically on a generalization of it. The complexity of computing the algebraic immunity of a Boolean function has been investigated in various works (*e.g.* [3, 15, 21, 30]). One approach, which to our knowledge has rarely been formally articulated and is often considered folklore, relies on submatrices of generator matrices of the Reed-Muller codes. We briefly recall its theoretical foundation here. The truth table of an n -variable Boolean function of degree at most r can be identified with a codeword of the Reed-Muller code $RM(r, n)$, where each codeword's element corresponds to the evaluation of a function at an element of $\mathbb{F}_2^n$. Thus, by puncturing the Reed-Muller code of order r , keeping only the elements x such that $f(x) = 1$, one obtains the code of the products of f with functions of degree at most r . The dimension of this code equals that of the Reed-Muller code of order r if and only if f does not admit a nonzero annihilator of degree at most d . The most efficient algorithm for computing the AI is the iterative method introduced in [3]. Rather than constructing submatrices of the generator matrix of a Reed-Muller code for successive values of d , this algorithm builds them incrementally by adding one row and one column at a time. Using the same theoretical foundation as the initial approach, a rank drop in the constructed matrices indicates the existence of an annihilator. This iterative process improves computational efficiency, and specific operations designed to reduce the cost of updating successive matrices improve the initial time complexity from $\mathcal{O}(\binom{n}{AI(f)}^\omega)$ (where $AI(f)$ denotes the AI of f) with $\omega = 3$ to a value closer to 2 [3, 30]. In this work, we focus on a generalization of algebraic immunity, namely the restricted algebraic immunity, introduced in [7]. This property considers a subset $S \subseteq \mathbb{F}_2^n$ and corresponds to the minimal degree of an annihilator g that annihilates f or $f + 1$ over the entire set S but is not identically zero on S . More formally, it is defined as: $AI_S(f) = \min_g (\deg(g) \mid fg = 0 \text{ or } g(f + 1) = 0, \text{ and } \exists x \in S \text{ such that } g(x) = 1)$. In this setting, classical algebraic immunity corresponds to the special case where $S = \mathbb{F}_2^n$. Since the Reed-Muller approach has not been formalized for this generalization³, and the iterative method has not been studied for this generalization, we address this gap by establishing the key properties that ensure the correctness of the adapted algorithms for restricted algebraic immunity, for any subset of $\mathbb{F}_2^n$. Additionally, we provide corresponding algorithms and code implementations.

Our contributions start in Section 3, where we formalize the Reed-Muller approach, and in Section 4, we analyze the iterative method, emphasizing the theoretical and practical challenges that arise when applying this technique to restricted sets. Variants of the algebraic attack, such as fast algebraic attacks [2, 14] and attacks leveraging monomials of extreme degrees [49], require computing parameters in a manner similar to restricted algebraic immunity. Accordingly, we hope that this work will be useful for further research in these directions, and that the new algorithms can be a basis for determinations of variants of the AI.

Next, we perform various experiments with our algorithms. First, we compare the practical efficiency of the two main approaches in terms of both runtime and memory usage. For this comparison, we consider algebraic immunity restricted to a slice of $\mathbb{F}_2^n$, specifically a set $E_{k,n}$ consisting of all binary vectors of length n with Hamming weight k . These sets, as well as functions that are balanced on all such sets, known as Weightwise (Almost) Perfectly Balanced (W(A)PB) functions, have attracted considerable attention since the introduction of restricted criteria in [7]. Slices of $\mathbb{F}_2^n$ frequently appear in cryptographic research, notably in contexts such as the Hamming weight model in side-channel analysis (where the Hamming weight of an intermediate variable, or a noisy version of it, can be learned by the adversary) and the fixed input weight setting in FLIP [47] (where by design the Hamming weight in input of the filtering function is known

³ in [7] some techniques to derive bounds on the restricted algebraic immunity are based on properties of the corresponding punctured Reed-Muller code, but no algorithm is given to determine the restricted algebraic immunity

by the adversary). In particular, WAPB functions have been extensively studied, with research focusing primarily on two aspects. The first is the construction of such functions, which has been explored in various works [7, 18, 19, 24, 27, 29, 34, 38, 41, 42, 45, 51, 52, 57, 60, 62, 63, 64, 65]. The second is the study of their cryptographic properties, including nonlinearity restricted to slices [23], overall nonlinearity [26], and algebraic immunity [25].

In this work, we further investigate algebraic immunity restricted to slices, denoted as Al_k , particularly in the context of WAPB functions, which are relevant when the Hamming weight of a function's input can be inferred. In Section 5.1, we analyze the distribution of Al_k for WPB functions with 4, 8, and even 16 variables. Then, in Section 5.2, we determine the values of Al_k for several families of functions introduced in previous works.

2 Preliminaries

For readability, we use the notation $+$ instead of $\oplus$ to denote the addition in $\mathbb{F}_2$, and $\sum$ instead of $\bigoplus$. In addition to classic notations, we denote by $[a, b]$ the subset of all integers between a and b : $\{a, a+1, \dots, b\}$. For a vector $v \in \mathbb{F}_2^n$ we use $w_H(v)$ to denote its Hamming weight $w_H(v) = |\{i \in [1, n] \mid v_i = 1\}|$. For two vectors v and w in $\mathbb{F}_2^n$ we denote by $d_H(v, w)$ the Hamming distance between v and w , that is, $d_H(v, w) = w_H(v + w)$. For two functions f and g we denote by $d_H(f, g)$ the Hamming distance between their vectors of values.

2.1 Boolean functions, cryptographic criteria, and weightwise properties

In this section, we recall fundamental concepts concerning Boolean functions and their weightwise properties, which are utilized throughout this article. For a more comprehensive introduction to Boolean functions and their cryptographic parameters, we recommend consulting the book by Carlet [5], and for insights into weightwise properties—also known as properties on the slices—the article [7]. We denote by $E_{k,n}$ the set $\{x \in \mathbb{F}_2^n \mid w_H(x) = k\}$ for $k \in [0, n]$, referring to it as a slice of the Boolean hypercube (of dimension n). Consequently, the Boolean hypercube is divided into $n+1$ slices, where the elements share the same Hamming weight.

Definition 1 (Boolean Function). A Boolean function f in n variables is a function from $\mathbb{F}_2^n$ to $\mathbb{F}_2$. The set of all Boolean functions in n variables is denoted by $\mathcal{B}_n$.

When a property or a definition is restricted to a slice, we denote it by using the subscript k . For example, for an n -variable Boolean function f we denote its support $\text{supp}(f) = \{x \in \mathbb{F}_2^n \mid f(x) = 1\}$. Furthermore, we denote by $\text{supp}_k(f)$ the support of f restricted to a slice, which is defined as $\text{supp}(f) \cap E_{k,n}$.

Definition 2 (Balancedness). A Boolean function $f \in \mathcal{B}_n$ is called balanced if $|\text{supp}(f)| = 2^{n-1} = |\text{supp}(f+1)|$.

For $k \in [0, n]$ the function is said balanced on the slice k if $||\text{supp}_k(f)| - |\text{supp}_k(f+1)|| \leq 1$. In particular when $|E_{k,n}|$ is even $|\text{supp}_k(f)| = |\text{supp}_k(f+1)| = |E_{k,n}|/2$.

Using the notion of restricted balancedness we can define the weightwise (almost) perfectly balanced functions, the focus of our work.

Definition 3 (Weightwise (Almost) Perfectly Balanced Function (WPB and WAPB)). Let $m \in \mathbb{N}^*$ and f be a Boolean function in $n = 2^m$ variables. It will be called weightwise perfectly balanced (WPB) if, for

every $k \in [1, n-1]$, f is balanced on the slice k , that is $\forall k \in [1, n-1], |\text{supp}_k(f)| = \binom{n}{k}/2$, and:

$$f(0, \dots, 0) = 0, \quad \text{and } f(1, \dots, 1) = 1.$$

The set of WPB functions in 2^m variables is denoted $\mathcal{WPB}_m$.

When n is not a power of 2, other weights than $k = 0$ and n can lead to slices of odd cardinality, we call $f \in \mathcal{B}_n$ weightwise almost perfectly balanced (WAPB) if:

$$|\text{supp}_k(f)| = \begin{cases} |E_{k,n}|/2 & \text{if } |E_{k,n}| \text{ is even,} \\ (|E_{k,n}| \pm 1)/2 & \text{if } |E_{k,n}| \text{ is odd.} \end{cases}$$

The set of WAPB functions in n variables is denoted $\mathcal{WAPB}_n$.

We first define additional fundamental concepts for the study of Boolean functions, in particular the algebraic normal form. We then introduce key cryptographic criteria for these functions, including algebraic immunity (both general and weightwise) and nonlinearity (both general and weightwise).

Definition 4 (Algebraic Normal Form (ANF) and degree). We call Algebraic Normal Form of a Boolean function f its n -variable polynomial representation over $\mathbb{F}_2$ (i.e. belonging to $\mathbb{F}_2[x_1, \dots, x_n]/(x_1^2 + x_1, \dots, x_n^2 + x_n)$):

$$f(x_1, \dots, x_n) = \sum_{I \subseteq [1, n]} a_I \left(\prod_{i \in I} x_i \right),$$

where $a_I \in \mathbb{F}_2$. The (algebraic) degree of f , denoted $\deg(f)$ is:

$$\deg(f) = \max_{I \subseteq [1, n]} \{|I| \mid a_I = 1\} \text{ if } f \text{ is not null, } 0 \text{ otherwise.}$$

We also recall the concept of order on $\mathbb{F}_2^n$, as it will be utilized in the algorithms involving Boolean functions.

Definition 5 (Order). A binary relation $\preceq$ on a set X is called partial order if $\preceq$ is reflexive, transitive and antisymmetric. Moreover, $\preceq$ is a total order if for all $a, b \in X$ it holds $a \preceq b$ or $b \preceq a$.

We give two examples of orders on n -length binary strings, more examples can be found in e.g. [54]:

- Lexicographic, given $a, b \in \mathbb{F}_2^n$ as $a = a_1, \dots, a_n$ and $b = b_1, \dots, b_n$, $a \preceq_{lex} b$ if and only if $a_i < b_i$ on the first index $i \in [1, n]$ such that $a_i \neq b_i$, or $a = b$.
- Graded lexicographic, given $a, b \in \mathbb{F}_2^n$ as $a = a_1, \dots, a_n$ and $b = b_1, \dots, b_n$, $a \preceq_{glex} b$ if $w_H(a) < w_H(b)$ or $w_H(a) = w_H(b)$ and $a \preceq_{lex} b$

Definition 6 (Truth table). Let $f \in \mathcal{B}_n$ be a Boolean function, its truth table is the Boolean vector defined as:

$$TT(f) \in \mathbb{F}_2^{2^n}, \quad TT(f) = (f(0_n), \dots, f(1_n)).$$

Denoting by f_S the restriction of f to a subset $S \subseteq \mathbb{F}_2^n$ written as $S = \{a_1, a_2, \dots, a_{|S|}\}$, the truth table of f_S refers to the vector $(f(a_1), f(a_2), \dots, f(a_{|S|}))$.

If not specified otherwise, the (restricted) truth table follows the lexicographic order.

Definition 7 (Algebraic immunity and restricted algebraic immunity). The algebraic immunity of a Boolean function $f \in \mathcal{B}_n$, denoted as $\text{Al}(f)$, is defined as:

$$\text{Al}(f) = \min_{g \neq 0} \{\deg(g) \mid fg = 0 \text{ or } (f+1)g = 0\},$$

where $\deg(g)$ is the algebraic degree of g . The function g is called an annihilator of f (or $f+1$).

The restricted algebraic immunity of a Boolean function $f \in \mathcal{B}_n$ on the set $S \subseteq \mathbb{F}_2^n$, denoted as $\text{Al}_S(f)$, is defined as:

$$\text{Al}_S(f) = \min_{g \neq 0 \text{ over } S} \{\deg(g) \mid fg = 0 \text{ or } (f+1)g = 0\}.$$

For $S = E_{k,n}$ we denote $\text{Al}_{E_{k,n}}(f)$ by $\text{Al}_k(f)$ and call it weightwise algebraic immunity.

Remark 1. Let $f \in \mathcal{B}_n$. We use the notation $f^{(o)}$ referring to either f or $f+1$ when the same process or result applies to any of them. In particular, $f^{(o)} \in \{f, f+1\}$.

2.2 Reed-Muller codes

Definition 8 (Monomials and set of monomials). Let $u, x \in \mathbb{F}_2^n$ denoted as $x = (x_1, \dots, x_n)$ and $u = (u_1, \dots, u_n)$, the monomial x^u is calculated as:

$$x^u = \prod_{i=1}^n x_i^{u_i}.$$

Let $E = \{\alpha_1, \alpha_2, \dots, \alpha_k\} \subseteq \mathbb{F}_2^n$, we denote by $x^E = \{x^{\alpha_1}, x^{\alpha_2}, \dots, x^{\alpha_k}\}$ the set of all monomials associated to E .

Definition 9 (Reed-Muller code - RM). Let $n \in \mathbb{N}^*$ and $r \in \mathbb{N}$ with $r \leq n$, the Reed-Muller code $RM(r, n)$ is the set of all 2^n -bit length binary vectors where each code word is the evaluation of one distinct Boolean function of degree at most r in n variables.

Definition 10 (RM generator matrix). Let X be the set of the 2^n elements in $\mathbb{F}_2^n$ in lexicographical order and $P_{r,n} = \{u \in \mathbb{F}_2^n \mid w_H(u) \leq r\}$ the set of all the elements of $\mathbb{F}_2^n$ with Hamming weight smaller than or equal to r . Let $D_r^n = \sum_{i=0}^r \binom{n}{i}$, the generator matrix $G_{r,n} \in \text{Mat}_{D_r^n, 2^n}$ is defined as

$$(G_{r,n})_{i,j} = x_j^{u_i}, \quad x_j \in X, \quad u_i \in P_{r,n}.$$

We denote by $G_{r,n}^Y \in \text{Mat}_{D_r^n, |Y|}$, the Reed-Muller generator matrix restricted on the set $Y \subseteq X$:

$$(G_{r,n})_{i,j}^Y = x_j^{u_i}, \quad x_j \in Y, \quad u_i \in P_{r,n}.$$

Remark 2. Let $\hat{a}$ be the vector corresponding to the coefficients in the ANF of a Boolean function f of n variables and degree r , then: $\hat{a} \cdot G_{r,n} = TT(f)$.

3 Computing the restricted AI from (punctured) generator matrices of Reed-Muller codes

In this section we provide an algorithm to compute the algebraic immunity of a Boolean function restricted on a general subset $S \subseteq \mathbb{F}_2^n$.

The algorithm aims to determine whether the systems of equations $S_1^r \triangleq g_1^r * f_S(x) = \hat{0}$ or $S_2^r \triangleq g_2^r * (f + 1)_S(x) = \hat{0}$ admit a non-zero solution for $x \in \mathbb{F}_2^n$, where g_1^r and g_2^r are Boolean functions of algebraic degree at most r . This is achieved by examining whether the rank of one of the punctured Reed-Muller (RM) matrices is smaller than one with the columns corresponding to the entire S . If f is non-constant on S , the algorithm searches for the existence of either g_1^r solving S_1^r or g_2^r solving S_2^r , where neither function vanishes over the entire set S . If such solutions g_1^r or g_2^r do not exist, new systems S_1^{r+1} and S_2^{r+1} are considered (through bigger matrices), and the search for solutions g_1^{r+1} and g_2^{r+1} continues. This iterative process is repeated until either an annihilator of f or $f + 1$ is found or an upper bound $r + \delta$ is reached. The procedure for f_S and $(f + 1)_S$ is carried out in parallel. Conversely, if f is constant on S , the algorithm terminates and returns 0, as justified by Proposition 1.

Proposition 1. *A Boolean function f has an AI_S of 0 if and only if the restriction f_S on S is constant.*

Proof. First, we show the reverse implication. We suppose that $f \in \mathcal{B}_n$ is constant on S , either $f(x) = 0$ for all $x \in S$, or $f(x) = 1$ for all $x \in S$. If f is not null everywhere but $f(x) = 0 \forall x \in S$, then the constant function $g(x) = 1$ is not zero on all S , and is it such that $g(x)f(x) = 0$ for all $x \in S$. Similarly, if for all $x \in S$ $f(x) = 1$, the same argument can be used for the function $f + 1$ on S .

Then, we show the direct implication. We suppose that a function $g \in \mathcal{B}_n$ is a non-zero-annihilator of f restricted to S of degree 0. Then, $g \neq 0$ and $\deg(g) = 0 \Rightarrow g = 1$, therefore since g annihilates f in S , $g(x)f(x) = 0$ for all $x \in S$, resulting in $f(x)g(x) = f(x) = 0$. Similarly, if g annihilates $f + 1$ on S instead of f , we have that $g(x)(f(x) + 1) = 0$ implying that $f + 1 = 0 \Rightarrow f(x) = 1$ for all $x \in S$. $\square$

The matrices corresponding to the systems of equations S_1^r and S_2^r are obtained by extracting the punctured Reed-Muller generator matrices, as defined in Definition 11, from the Reed-Muller generator matrix, considering only the columns corresponding to elements in S .

Definition 11. *Let $r, n \in \mathbb{N}^*$ with $r \leq n$. Let $G_{r,n}$ be the generator matrix of the Reed-Muller code $\text{RM}(r, n)$, we define the object $G_{r,n}^{f_S^{(o)}, S}$ by setting to zero all the columns of $G_{r,n}^S$ which do not correspond to elements of $\text{supp}(f^{(o)}) \cap S$. For the remaining columns, we calculate $G_{r,n}^{f_S^{(o)}, S}$ as*

$$\left(G_{r,n}^{f_S^{(o)}, S} \right)_{i,j} = f_S^{(o)}(u_j) \cdot (G_{r,n}^S)_{i,j}, \quad u_j \in S$$

Moreover, we define the object $G_{r,n}^{f_{\text{supp}(f^{(o)}) \cap S}^{(o)}, S}$, by taking all the columns of $G_{r,n}$ which corresponds to element of $\mathbb{F}_2^n$ which are in $\text{supp}(f^{(o)}) \cap S$.

$$\left(G_{r,n}^{f_{\text{supp}(f^{(o)}) \cap S}^{(o)}, S} \right)_{:,j} = (G_{r,n}^S)_{:,j} \text{ if } f_S^{(o)}(u_j) = 1 \quad (1)$$

Proposition 2. *Let $Z = \text{supp}(f^{(o)}) \cap S$. If $\text{rank} \left(G_{r,n}^{f_Z^{(o)}, S} \right) < \text{rank}(G_{r,n}^S)$, then $f^{(o)}$ admits a non-zero-annihilator restricted to S of degree at most r .*

Proof. Let $D_r^n = \sum_{i=0}^r \binom{n}{i}$. Since $D_r^n = \text{rank}(G_{r,n}^{f^{(o)},S}) + |\text{Ker}(G_{r,n}^{f^{(o)},S})| = \text{rank}(G_{r,n}^S) + |\text{Ker}(G_{r,n}^S)|$, we have that $\text{rank}(G_{r,n}^{f^{(o)},S}) < \text{rank}(G_{r,n}^S) \iff |\text{Ker}(G_{r,n}^{f^{(o)},S})| > |\text{Ker}(G_{r,n}^S)|$. Therefore $\exists v \in \text{Ker}(G_{r,n}^{f^{(o)},S})$ such that $v \notin \text{Ker}(G_{r,n}^S)$. Hence, it exists a Boolean function g of degree r , whose truth table is given by $v \cdot G_{r,n}$, which is such that $v \cdot G_{r,n} \neq 0$ and $v \cdot G_{r,n}^S \neq 0$, but $(g \cdot f_S^{(o)})_S = v \cdot G_{r,n}^{f^{(o)},S} = 0$. Since $\text{rank}\left(G_{r,n}^{f_S^{(o)},S}\right) = \text{rank}\left(G_{r,n}^{f^{(o)},S}\right)$, $\text{rank}(G_{r,n}^{f^{(o)},S}) < \text{rank}(G_{r,n}^S) \Rightarrow \text{rank}(G_{r,n}^{f_S^{(o)},S}) < \text{rank}(G_{r,n}^S)$ and hence the condition $\text{rank}(G_{r,n}^{f_Z^{(o)},S}) < \text{rank}(G_{r,n}^S)$ is enough to justify the existence of such v giving the non-zero-annihilator g of f^o restricted to S . $\square$

Algorithm 3, provided in Appendix B, details the full procedure to determine the restricted algebraic immunity of a function f of n variables on the subset S . We give additional details:

Remark 3.

1. [7] established that the upper bound of $\text{AI}_S(f)$ for any Boolean function f and any set $S \subseteq \mathbb{F}_2^n$, is the smallest integer r such that $\binom{n}{r} > |S|/2$. As a consequence, the search of the annihilator for f and $f + 1$ in our algorithm terminates once $\text{rank}(G_{r,n}^S) > \lfloor |S|/2 \rfloor$.
2. The generator matrix of a Reed-Muller code does not depend on the Boolean function. It is therefore possible to pre-compute and store the generator matrices of all Reed-Muller codes with r and n smaller than some maximum values r_{max} and n_{max} . In particular, when computing AI_S of a function with nn variables using Algorithm 3, the pre-computation of $G_{n,n}$ is required. However, to facilitate the use of the algorithm for functions with fewer than n variables, we also pre-compute matrices for smaller values of n . We therefore fix an n_{max} and pre-compute the hash table

$$G^{\leq n_{max}} = \{(n, G_{n,n})\}_{1 \leq n \leq n_{max}} \quad (2)$$

and store it in a file.

Similarly, we can also pre-compute and store a second hash table, containing all the D^n s such that $n \leq n_{max}$. Such hash table is

$$D^{\leq n_{max}} = \{(n, D^n)\}_{0 \leq n \leq n_{max}} = \{(n, \{(n, D_i^n)\}_{0 \leq i \leq n})\}_{0 \leq n \leq n_{max}} \quad (3)$$

In Algorithm 3, we therefore use the function `READ_VALUES_FROM_FILE`, to read to two pre-computed hash tables from the files where they have been pre-stored before the execution of the algorithm.

3. The procedure `FIND_DEG_SMALLEST_S_ANNIHILATOR` in Algorithm 3 is performed in parallel for f and $f + 1$ with the additional condition that if one of the two branches returns the value 1, the other branch is stopped, without having to wait for it to finish.

4 Computing the restricted AI iteratively

In this section, we present an algorithm (Algorithm 1) to determine the algebraic immunity restricted to a set $S \subseteq \mathbb{F}_2^n$ with improved time and space complexity compared to the method presented in Section 3.

The core idea is to iteratively construct the objects $G_{r,n}^{f_{\text{supp}(f) \cap S}, S}$ and $G_{r,n}^{(f+1)_{\text{supp}(f+1) \cap S}, S}$ for varying r , increasing the size of the matrix by one row per iteration while minimizing computations. This approach

checks for the existence of annihilators dynamically, rather than extracting them from a large, pre-computed Reed-Muller generator matrix restricted to S . The method extends the approach of [3] for determining the standard algebraic immunity. As with Algorithm 3, this procedure returns 0 if the function f is constant on S . Hence, we focus on explaining the case of determining $\text{Al}_S(f)$ for functions such that $\text{Al}_S(f) \geq 1$. (The full algorithm, however, also addresses functions with null Al_S .)

We begin with definitions and remarks necessary to introduce the general idea of the algorithm.

Definition 12 (T-Vandermonde matrix). Let $Z = \{z_1, \dots, z_{k_Z}\} \subseteq \mathbb{F}_2^n$ and $E = \{\alpha_1, \dots, \alpha_{k_E}\} \subseteq \mathbb{F}_2^n$, we define the matrix $V_{E,Z}$ whose entries are

$$(V_{E,Z})_{i,j} = z_j^{\alpha_i} \quad \forall i \in [1, k_E] \text{ and } j \in [1, k_Z].$$

We refer to this matrix as the T-Vandermonde matrix, as it is the transpose of a Vandermonde matrix. Denoting by $Z_j = \{z_1, z_2, \dots, z_j\}$ and $E_i = \{\alpha_1, \dots, \alpha_i\}$ for $j \in [1, k_Z]$ and $i \in [1, k_E]$, the sub-matrix V_{E_i, Z_j} of $V_{E,Z}$, is given by the elements:

$$(V_{E_i, Z_j})_{m,l} = z_l^{\alpha_m} \quad \forall m \in [1, i] \text{ and } l \in [1, j].$$

Similarly, we define the matrices $S_{E,S}$ and S_{E_i, S_j} as the T-Vandermonde sub-matrices corresponding to the elements of the set $S \subseteq \mathbb{F}_2^n$.

Remark 4. Let $Z = \text{supp}(f) \cap S$ and $E = \{\alpha_1, \alpha_2, \dots, \alpha_{D_d^n}\}$ be such that x^E is the set of all monomials with degree at most d ordered by the graded lex order. We remark the following:

- $V_{E,Z} = G_{d,n}^{f_Z, S}$.
- Let $E^* \subset E$, ordered by graded lex order. $V_{E^*, Z}$ is a sub-matrix of $G_{d,n}^{f_Z, S}$.

Remark 5. Let $i \geq 1$. If $1 < i+1 \leq \min(|Z|, |E_{D_d^n}|)$, then $V_{E_{i+1}, Z_{i+1}}$ can be computed as

$$V_{E_{i+1}, Z_{i+1}} = \begin{pmatrix} & & & z_{i+1}^{\alpha_1} \\ & & & z_{i+1}^{\alpha_2} \\ & & & \vdots \\ & & & z_{i+1}^{\alpha_i} \\ V_{E_i, Z_i} & & & z_{i+1}^{\alpha_{i+1}} \\ z_1^{\alpha_{i+1}} & z_2^{\alpha_{i+1}} & \dots & z_i^{\alpha_{i+1}} & z_{i+1}^{\alpha_{i+1}} \end{pmatrix}.$$

This can be seen by combining Definition 12 and the fact that for all i such that $1 < i+1 \leq \min(|Z|, |E| - 1)$, the matrix $V_{E_{i+1}, Z_{i+1}}$ is square.

Definition 13 (Sequence of V_k matrices). As a consequence of Remark 5, we define the sequence of sub-matrices $(V_k)_{k \in [1, D_d^n]}$ by

$$V_k = \begin{cases} \begin{pmatrix} & & & z_k^{\alpha_1} \\ & & & z_k^{\alpha_2} \\ & & & \vdots \\ & & & z_k^{\alpha_{k-1}} \\ V_{k-1} & & & z_k^{\alpha_k} \\ z_1^{\alpha_k} & z_2^{\alpha_k} & \dots & z_{k-1}^{\alpha_k} & z_k^{\alpha_k} \end{pmatrix} & \text{if } 2 \leq k \leq \min(|Z|, |E_{D_d^n}|), \\ \begin{pmatrix} & & & z_k^{\alpha_1} \\ & & & z_k^{\alpha_2} \\ & & & \vdots \\ & & & z_k^{\alpha_{k-1}} \\ V_{k-1} & & & z_k^{\alpha_k} \\ z_1^{\alpha_k} & z_2^{\alpha_k} & \dots & z_k^{\alpha_k} \end{pmatrix} & \text{if } |Z| < k \leq |E_{D_d^n}|, \end{cases}$$

with $V_1 = \begin{pmatrix} z_1^{\alpha_1} \end{pmatrix} = z_1^{0_n} = (1)$.

With Definition 12 and Remark 4 in mind, we describe the algorithm's procedure as follows. Let $Z = \text{supp}(f^{(o)}) \cap S$, and let $1 \leq i \leq |Z|$. If the function f is not constant on S , the algorithm begins by constructing the matrix V_{E_i, Z_i} and searches for functions vanishing on the set Z_i by identifying vectors corresponding to the elements of the kernel of V_{E_i, Z_i} . If such functions exist, the procedure checks whether any of them are also annihilators restricted to S for the full function $f^{(o)}$. This is done by verifying that they vanish on all elements of Z but not on all elements of S . If such an annihilator is found, the algorithm returns the degree of the last monomial added, x^{α_i} . Otherwise, it increments the index i , constructs the next matrix following Definition 13, and continues until a solution is found or all square matrices up to $V_{E_i, |Z|}$ have been constructed and tested. If no annihilator is found after all the elements of Z have been added, the algorithm iteratively adds new monomials and generates the matrices $V_{E_i, Z}$ and S_{E_i, S_i} . It then returns the degree of the monomial x^{α_i} if $\text{rank}(V_{E_i, Z}) < \text{rank}(S_{E_i, S_i})$. Following point 1 in Remark 3, if this condition is not satisfied at step i , the iteration continues until $\text{rank}(S_{E_i, S_i}) > |S|/2$. Indeed, if for some i , $\text{rank}(V_{E_i, Z}) < \text{rank}(S_{E_i, S_i})$, then there exists a non-null annihilator over S of degree at most $\deg(\alpha_i)$. Conversely, if the condition is not met before $\text{rank}(S_{E_i, S_i}) > |S|/2$, Remark 3 guarantees that either f or $f + 1$ admits a non-trivial annihilator on S of this degree.

4.1 Keeping the cardinality of the kernel of V_k upper-bounded by 1

In the procedure described, we aim to identify at least one element from a vector space that satisfies certain required conditions (corresponding to an annihilator, not trivial on S), while maintaining the algorithm's desired efficiency. Since this vector space can potentially be too large, we address this issue as follows: if we find an element $\hat{g}_k$ in the kernel of V_k that does not vanish on Z , we re-order the vector Z by swapping the element z_{k+1} with the element that does not vanish in $\hat{g}_k$. This ensures that such elements of the kernel of V_k do not propagate into the kernel of subsequent matrices.

Furthermore, if an element of the kernel of V_k is found that vanishes on all of Z and S , the monomial α_k is dropped. The matrix V_{k+1} is then constructed by replacing the row corresponding to the monomial α_k with one corresponding to the monomial α_{k+1} . This procedure guarantees that the cardinality of the kernel of V_k is upper-bounded by 1 for all $k \leq |Z|$. Propositions 3, 4, and 5 provide formal justifications for these claims.

Proposition 3. *Let $k < \min\{|Z|, |E_{D_n^n}|\}$ and let V_k constructed following Definition 13. Suppose $\ker(V_k) = \langle \hat{g}_k \rangle = \langle (\epsilon_1, \epsilon_2, \dots, \epsilon_k) \rangle$ and let $g_k \in \mathcal{B}_n$ be the Boolean function having $\hat{g}_k$ as ANF. Furthermore, assume there exists a z_{k+j} , for $j \geq 1$ such that $g_k(z_{k+j}) = 1$. Then, the matrix V'_{k+1} constructed as:*

$$V'_{k+1} = \begin{pmatrix} & & & z_{k+j}^{\alpha_1} \\ & & & z_{k+j}^{\alpha_2} \\ & & & \vdots \\ & & & z_{k+j}^{\alpha_k} \\ & & & z_{k+j}^{\alpha_{k+1}} \\ z_1^{\alpha_{k+1}} & z_2^{\alpha_{k+1}} & \dots & z_k^{\alpha_{k+1}} \end{pmatrix}$$

is such that $(\epsilon_1, \epsilon_2, \dots, \epsilon_k, 0) \notin \ker(V'_{k+1})$.

Proof. Suppose $\ker(V_k) = \langle \hat{g}_k \rangle = \langle (\epsilon_1, \epsilon_2, \dots, \epsilon_k) \rangle$, and that there exists $j \geq 1$ such that $g_k(z_{k+j}) = 1$, but it is also such that $(\epsilon_1, \epsilon_2, \dots, \epsilon_k, 0) \in \ker(V'_{k+1})$.

If $(\epsilon_1, \epsilon_2, \dots, \epsilon_k, 0) \in \ker(V'_{k+1})$, then $(\epsilon_1, \epsilon_2, \dots, \epsilon_k, 0) \cdot V'_{k+1} = \hat{0}$. Specifically, the last equation in the system of equations described by $(\epsilon_1, \epsilon_2, \dots, \epsilon_k, 0) \cdot V'_{k+1} = \hat{0}$ is:

$$\epsilon_1 z_{k+j}^{\alpha_1} + \dots + \epsilon_k z_{k+j}^{\alpha_k} + 0 \cdot z_{k+j}^{\alpha_{k+1}} = \epsilon_1 z_{k+j}^{\alpha_1} + \dots + \epsilon_k z_{k+j}^{\alpha_k} = 0.$$

However, from our hypothesis, we have:

$$g_k(z_{k+j}) = 1 \iff g_k(z_{k+j}) = (\epsilon_1, \epsilon_2, \dots, \epsilon_k) \begin{pmatrix} z_{k+j}^{\alpha_1} \\ z_{k+j}^{\alpha_2} \\ \vdots \\ z_{k+j}^{\alpha_k} \end{pmatrix} = \epsilon_1 z_{k+j}^{\alpha_1} + \dots + \epsilon_k z_{k+j}^{\alpha_k} = 1.$$

This contradiction shows that it cannot be the case that $(\epsilon_1, \dots, \epsilon_k, 0)$ is in the kernel of V'_{k+1} . □

Proposition 4. *The sequence of matrices $(V_k)_k$, constructed as described in Proposition 3, ensures that:*

$$\dim(\ker(V_k)) \leq 1, \quad \forall k \in [1, |Z|].$$

Proof. We prove the result by induction:

- Base case. The initial matrix $V_1 = (1)$ has full rank, so $\dim(\ker(V_1)) = 0 \leq 1$.
- Induction step. We suppose that $\dim(\ker(V_k)) \leq 1$. We show that $\dim(\ker(V_{k+1})) \leq 1$.
 - If $\dim(\ker(V_k)) = 0$, adding a new row and column to form V_{k+1} can increase the kernel by at most 1. Hence, $\dim(\ker(V_{k+1})) \leq 1$.
 - If $\dim(\ker(V_k)) = 1$, let $\hat{z}$ be an element of $\ker(V_{k+1})$. Then either:
 - * $\hat{z} = (\hat{\epsilon}, 0)$:

$$\hat{z} \cdot V_{k+1} = 0 \implies \hat{\epsilon} \cdot V_k = 0 \implies \hat{\epsilon} \in \ker(V_k).$$

By Proposition 3, $\hat{\epsilon}$ can only be a trivial solution.

- * $\hat{z} = (\hat{y}, 1)$, where $\hat{y} \in \mathbb{F}_2^k$: In this case $\hat{z} \cdot V_{k+1} = 0$ implies:

$$\hat{y} \cdot V_k + (z_1^{\alpha_{k+1}}, \dots, z_k^{\alpha_{k+1}}) = 0 \iff \hat{y} \cdot V_k = (z_1^{\alpha_{k+1}}, \dots, z_k^{\alpha_{k+1}}), \quad (4)$$

and

$$\hat{y} \cdot (z_{k+1}^{\alpha_1}, \dots, z_{k+1}^{\alpha_k})^T + z_{k+1}^{\alpha_{k+1}} = 0 \iff \hat{y} \cdot (z_{k+1}^{\alpha_1}, \dots, z_{k+1}^{\alpha_k})^T = z_{k+1}^{\alpha_{k+1}}. \quad (5)$$

Since $\dim(\ker(V_k)) = 1$, $\text{rank}(V_k) = k - 1$. Hence, there exists at most one particular solution $\hat{y}_p$ for Equation 4. If $\hat{y}_p$ also solves Equation 5, then $\hat{y}_p$ is in $\ker(V_{k+1})$.

Since $(\hat{\epsilon}, 0)$ cannot be in $\ker(V_{k+1})$, the only possible element in $\ker(V_{k+1})$ is $\hat{y}_p$, if it exists. Therefore, $\dim(\ker(V_{k+1})) \leq 1$. □

Remark 6. Notice that the definition of V'_{k+1} in Proposition 3, does not conflict with Definition 13, as constructing the sequence $(V_k)_k$ does not require a specific order of the elements in the support Z .

Proposition 5. Let $\ker(V_{E_k, Z_k}) = \langle \hat{g}_k \rangle = \langle (\epsilon_1, \dots, \epsilon_k) \rangle$, where the corresponding Boolean function $g_k \in \mathcal{B}_n$ satisfies $g_k(x) = 0, \forall x \in Z$.

Define $E^* = (\alpha_1, \alpha_2, \dots, \alpha_{k-1}, \alpha_{k+1})$, a vector constructed from E_k by replacing the last element α_k with α_{k+1} . Let $V_{k+1} = V_{E_{k+1}^*, Z_k}$ be the matrix defined by replacing the monomial α_k in V_{E_k, Z_k} , with the monomial α_{k+1} :

$$V_{E_{k+1}^*, Z_k} = \begin{pmatrix} & & & z_k^{\alpha_1} \\ & & & z_k^{\alpha_2} \\ & & & \vdots \\ & E_{k-1}, V_{Z_{k-1}} & & z_k^{\alpha_{k-1}} \\ z_1^{\alpha_{k+1}} & z_2^{\alpha_{k+1}} & \dots & z_{k-1}^{\alpha_{k+1}} & z_k^{\alpha_{k+1}} \end{pmatrix}.$$

Then, either $\hat{g}_k \notin \ker(V_{E_{k+1}^*, Z_k})$, or $\hat{g}_k$ is the only element (aside from $\hat{0}$) in the kernel of $V_{E_{k+1}^*, Z_k}$. Hence:

$$\dim(\ker(V_{E_{k+1}^*, Z_k})) \leq 1.$$

Proof. Proposition 3 ensures that the kernel of $V_{E_{k+1}^*, Z_k}$ does not contain elements derived from the kernel of $V_{E_{k-1}, Z_{k-1}}$. Therefore, $V_{E_{k-1}, Z_{k-1}}$ can be treated as a full-rank matrix. Modifying the last row of a matrix that contains a full-rank sub-matrix does not increase the size of the kernel. This is because the first k entries of the elements in the kernel of V_{E_k, Z_k} and $V_{E_{k+1}^*, Z_k}$ are the same, with the only difference arising from the last row. Since this row is the only one that can introduce linear dependence, the dimension of the kernel of $V_{E_{k+1}^*, Z_k}$ is bounded by 1. $\square$

4.2 Using column echelon form matrices of the previous steps

At the iteration k , the matrix V_k already contains all the columns of the matrix from the previous step, V_{k-1} . This means the column echelon form of V_{k-1} is already embedded within V_k . Therefore, instead of recalculating the column echelon form for the entire matrix V_k , we only need to update it for the newly added column. This approach saves computation because we are focusing only on the last column rather than reprocessing the whole matrix.

Definition 14 (Column Echelon form). Let $k_r, k_c \in \mathbb{N}^*$ and $M \in \text{Mat}_{k_r, k_c}$. We denote by $EF : \text{Mat}_{k_r, k_c} \rightarrow \text{Mat}_{k_r, k_c}$, the function transforming a matrix to its column echelon form.

For each iteration k of the algorithm, the matrix V_k already includes the sub-matrix V_{k-1} . Thus, it is possible to compute the column echelon form of V_{k-1} and use it to derive the column echelon form of V_k . This approach allows V_k to inherently contain $EF(V_{k-1})$, which is already in column echelon form, thereby reducing the computational steps required to determine the rank of V_k . However, to apply this method effectively, careful attention must be paid during the construction of the matrix. Since rows are appended to the matrix at each step, computing $EF(V_k)$ requires keeping track of all column operations performed while computing $EF(V_j)$ for all $j < k$. These operations must be applied before computing $EF(V_k)$. Remark 7 and Proposition 6 provide further justification for this claim.

Definition 15 (Equality up to column permutation). Two matrices M_1 and M_2 are equal up-to-permutation if there exists a column permutation ρ such that

$$M_1 = \rho M_2.$$

We denote it as $M_1 =_{\rho} M_2$

Remark 7. Let $k_r, k_c \in \mathbb{N}$ and $M \in \text{Mat}_{k_r, k_c}(\mathbb{F}_2)$. Let $M = L \cdot U$ where L a lower triangular matrix and U is an upper triangular matrix, then: $EF(M) =_p L$.

Proposition 6. Let k be such that $1 < k < \min(|Z|, |E_{D_n^n}|)$ it holds that

$$EF(V_k) =_p EF \left(\begin{pmatrix} & z_k^{\alpha_1} \\ & z_k^{\alpha_2} \\ & \vdots \\ & z_k^{\alpha_{k-1}} \\ (z_1^{\alpha_k} \ z_2^{\alpha_k} \ \dots \ z_{k-1}^{\alpha_k}) P_{k-1} & z_k^{\alpha_k} \end{pmatrix} \right).$$

where P_{k-1} is such that $U_{k-1}^{-1} = P_{k-1}$, where $V_{k-1} = L_{k-1} U_{k-1}$ with $EF(V_{k-1}) =_p L_{k-1}$

Proof. Let k be such that $1 < k < \min(|Z|, |E_{D_n^n}|)$. The LU -decomposition of V_{k-1} is $V_{k-1} = L_{k-1} U_{k-1}$, where U_{k-1} is an upper triangular matrix and $L_{k-1} = EF(V_{k-1})$ is the lower triangular matrix.

$$V_k = \begin{pmatrix} & z_k^{\alpha_1} \\ & z_k^{\alpha_2} \\ & \vdots \\ & z_k^{\alpha_{k-1}} \\ z_1^{\alpha_k} \ z_2^{\alpha_k} \ \dots \ z_{k-1}^{\alpha_k} & z_k^{\alpha_k} \end{pmatrix} = \begin{pmatrix} & z_k^{\alpha_1} \\ & z_k^{\alpha_2} \\ & \vdots \\ & z_k^{\alpha_{k-1}} \\ (z_1^{\alpha_k} \ z_2^{\alpha_k} \ \dots \ z_{k-1}^{\alpha_k}) U_{k-1}^{-1} & z_k^{\alpha_k} \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 0 \ 0 \ \dots \ 0 \ 1 \end{pmatrix}.$$

Since U_{k-1} is an upper triangular matrix, $\begin{pmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 0 \ 0 \ \dots \ 0 \ 1 \end{pmatrix}$ is also an upper triangular matrix, and therefore

$$EF(V_k) =_p EF \left(\begin{pmatrix} & z_k^{\alpha_1} \\ & z_k^{\alpha_2} \\ & \vdots \\ & z_k^{\alpha_{k-1}} \\ (z_1^{\alpha_k} \ z_2^{\alpha_k} \ \dots \ z_{k-1}^{\alpha_k}) U_{k-1}^{-1} & z_k^{\alpha_k} \end{pmatrix} \right).$$

Taking $L_{k-1} =_p EF(V_{k-1})$ and $P_{k-1} = U_{k-1}^{-1}$:

$$EF(V_k) =_p EF \left(\begin{pmatrix} & z_k^{\alpha_1} \\ & z_k^{\alpha_2} \\ & \vdots \\ & z_k^{\alpha_{k-1}} \\ (z_1^{\alpha_k} \ z_2^{\alpha_k} \ \dots \ z_{k-1}^{\alpha_k}) P_{k-1} & z_k^{\alpha_k} \end{pmatrix} \right).$$

□

Remark 8. P_{k-1} is the matrix corresponding to the history of operations applied to the columns of V_{k-1} to be able to obtain $EF(V_{k-1})$.

As a consequence of Remark 8, it can be seen that when calculating the column echelon form of the matrix V_k using P_{k-1} , we construct the application P_k which can be used to compute the column echelon form of V_{k+1} in the next iteration. Remark 9 shows this more formally.

Remark 9.

$$EF(V_k) = EF \left(\begin{pmatrix} EF(V_{k-1}) & \begin{matrix} z_k^{\alpha_1} \\ z_k^{\alpha_2} \\ \vdots \\ z_k^{\alpha_{k-1}} \end{matrix} \\ (z_1^{\alpha_k} \ z_2^{\alpha_k} \ \dots \ z_{k-1}^{\alpha_k})P_{k-1} & z_k^{\alpha_k} \end{pmatrix} \right) =_p \begin{pmatrix} EF(V_{k-1}) & \begin{matrix} 0 \\ 0 \\ \vdots \\ 0 \end{matrix} \\ (z_1^{\alpha_k} \ z_2^{\alpha_k} \ \dots \ z_{k-1}^{\alpha_k} \ z_k^{\alpha_k})P_k \end{pmatrix}.$$

Algorithm 1 shows the procedure to calculate the degree of the minimal degree non-zero-annihilator restricted to $S \subseteq \mathbb{F}_2^n$ of a function $f^{(o)} \in \{f, f+1\}$, while Algorithm 2, provided in Appendix B, shows the full procedure to determine the restricted algebraic immunity of f in S , calling Algorithm 1 for f and $f+1$ in parallel.

Algorithm 2 includes the additional function `FIND_DEG_SMALLEST_S_ANNIHILATOR_SEQ` which triggers the same process as Algorithm 1, but applies it sequentially to f and $f+1$, to avoid waiting for both processes to complete simultaneously. This function is triggered only when $\text{supp}(f) \cap S$ and $\text{supp}(f+1) \cap S$ differ significantly in cardinality, as in such cases, the probability of one of the two processes terminating quickly is high.

Remark 10. To keep the pseudocode in line with the theory, the elements in the vector or sets are often parsed by name instead of by index, namely let $Z = (z_1, z_2, \dots, z_k)$ we consider that $Z[i-1] = z_i$. Therefore, we have to clarify our notation in the case in which two vector elements get swapped. In the case of Algorithm 1, we consider that the swap generates a new vector with the name of the variables adapted with their index position in the vector to keep satisfying the condition $Z[i-1] = z_i$. Meaning that

$$\text{SWAP}(z_i, z_j) \rightarrow (z_{\sigma(1)}, \dots, z_{\sigma(i)}, \dots, z_{\sigma(j)}, \dots, z_{\sigma(k)}) = Z' = (z'_1, \dots, z'_i, \dots, z'_j, \dots, z'_k)$$

where σ is a permutation describing the swap:

$$\sigma = \begin{pmatrix} 1 \dots i \dots j \dots k \\ 1 \dots j \dots i \dots k \end{pmatrix}.$$

Algorithm 1 Algorithm to find the degree of the minimum-degree non-zero-annihilator restricted on S of a function $f^{(o)}$.

Input: restricted set $S \subseteq \mathbb{F}_2^n$, $Z = \text{supp}(f^{(o)}) \cap S$ and $E = \{\alpha_1, \dots, \alpha_{D_n}\}$

Output: Degree of minimal degree non-zero-annihilator restricted on S of $f^{(o)}$.

```

1:  $V \leftarrow (1)$ ;
2:  $ops \leftarrow []$ ;
3:  $k \leftarrow 2$ ;
4: while  $k \leq |Z|$  do
5:    $V_{previous} \leftarrow V$ ;
6:    $(v_1, \dots, v_{k-1}) \leftarrow \text{APPLY\_OPERATIONS}((z_1^{\alpha_k}, \dots, z_{k-1}^{\alpha_k}), ops)^4$ ;
7:    $M \leftarrow \begin{pmatrix} & & & z_k^{\alpha_1} \\ & & & z_k^{\alpha_2} \\ & & & \vdots \\ & & & z_k^{\alpha_{k-1}} \\ V & & & z_k^{\alpha_k} \\ v_1 & v_1 & \dots & v_{k-1} & z_k^{\alpha_k} \end{pmatrix}$ ;
8:    $V, ops_k \leftarrow \text{EF\_LAST\_COL}(M)^5$ ;
9:   if  $\text{rank}(V) < k$  then
10:      $k \leftarrow \text{kernel}(V).base()$ ;
11:      $\hat{g} \leftarrow k[0]$ ;
12:      $\text{nonVanishingPointIndex}, \text{vanishingFlag} \leftarrow \text{Null}, \text{True}$ ;
13:     for all  $z_{idx} \in (Z \setminus Z_k)$  do
14:        $\text{vanishCheck} \leftarrow \text{VANISHES\_ON}(\hat{g}, z_{idx})^6$ ;
15:       if  $\text{vanishCheck} = \text{False}$  then
16:          $\text{vanishingFlag} \leftarrow \text{False}$ ;
17:          $\text{nonVanishingPointIndex} \leftarrow idx$ 
18:       break;
19:     end if
20:   end for
21:   if  $\text{vanishingFlag} = \text{True}$  then
22:     for all  $z_c \in S \setminus Z$  do
23:        $\text{vanishCheckOnS} \leftarrow \text{VANISHES\_ON}(\hat{g}, z_c)$ ;
24:       if  $\text{vanishCheckOnS} = \text{False}$  then
25:         return  $w_H(\alpha_i)$ ;
26:       end if
27:     end for
28:      $V \leftarrow V_{previous}$ ;
29:      $\text{REMOVE}(\alpha_i, E)$ ; // Remove  $\alpha_i$  from the array  $E$ .
30:     continue; // Go to the next iteration.
31:   else
32:      $zIndex \leftarrow k + \text{nonVanishingPointIndex}$ ;
33:      $Z \leftarrow \text{SWAP}(z_{k+1}, z_{zIndex})^7$ ;
34:   end if
35:    $ops \leftarrow ops \parallel ops_k$ ;
36:    $k \leftarrow k + 1$ ;
37: end if
38: end while
39:  $S, ops_S \leftarrow \text{EF} \left( \begin{pmatrix} s_1^{\alpha_1} & s_2^{\alpha_1} & \dots & s_{|S|}^{\alpha_1} \\ \vdots & \vdots & \ddots & \vdots \\ s_1^{\alpha_k} & s_2^{\alpha_k} & \dots & s_{|S|}^{\alpha_k} \end{pmatrix} \right)$ ;
40:  $r \leftarrow \text{rank}(S)$ ;
41:  $r_V \leftarrow \text{rank}(V)$ ;
42: if  $r_V < r$  then
43:   return  $w_H(\alpha_k)$ ;
44: end if
45: // The algorithm continues in the next page.

```

```

46: while  $r \leq \lfloor |S|/2 \rfloor$  do
47:   if  $r_V < k$  then
48:      $(v_1, \dots, v_{k-1}) \leftarrow \text{APPLY\_OPERATIONS}((z_1^{\alpha_k}, \dots, z_{k-1}^{\alpha_k}), ops)$ 
49:      $V \leftarrow \begin{pmatrix} & & V \\ v_1 & v_2 & \dots & v_{k-1} & z_k^{\alpha_k} \end{pmatrix};$ 
50:      $r_V \leftarrow \text{rank}(V);$ 
51:   end if
52:    $(v_1, \dots, v_{k-1}) \leftarrow \text{APPLY\_OPERATIONS}((s_1^{\alpha_k}, \dots, s_{k-1}^{\alpha_k}), ops_S);$ 
53:    $S \leftarrow \begin{pmatrix} & & S \\ v_1 & v_2 & \dots & v_{k-1} & s_k^{\alpha_k} \end{pmatrix};$ 
54:    $r \leftarrow \text{rank}(S);$ 
55:   if  $r_V < r$  then
56:     return  $w_H(\alpha_k);$ 
57:   end if
58: end while

```

Algorithm 2 Algorithm to compute the algebraic immunity of a function $f \in \mathcal{B}_n$ restricted to S

Input: $x \in \mathbb{F}_2^{2^n}$ evaluation of the truth table of $f \in \mathcal{B}_n$ and the restricted set $S \subseteq \mathbb{F}_2^n$.

Output: $\text{Al}_S(f)$

```

1:  $Z, Z_c \leftarrow \text{supp}(x) \cap S, \text{supp}(x+1) \cap S;$ 
2: if  $Z = \emptyset$  or  $Z_c = \emptyset$  then return 0;
3: end if
4:  $E_{D_n^n} \leftarrow \text{CONSTRUCT\_MONOMIALS}(n = n, d = n)^8;$ 
5:  $r \leftarrow \frac{|Z|}{|Z_c|}$  if  $|Z| < |Z_c|$  else  $\frac{|Z_c|}{|Z|};$ 
6: if  $r \geq \frac{1}{4}$  then
7:    $\text{immunity}_f, \text{immunity}_{f+1} \leftarrow ($ 
8:    $\text{FIND\_DEG\_SMALLEST\_S\_ANNIHILATOR}(Z = Z, E = E_{D_n^n}, S = S)$ 
9:    $|\text{FIND\_DEG\_SMALLEST\_S\_ANNIHILATOR}(Z = Z_c, E = E_{D_n^n}, S = S)|);$ 
10:  return  $\text{MIN}(\text{immunity}_f, \text{immunity}_{f+1});$ 
11: else
12:  return  $\text{FIND\_DEG\_SMALLEST\_S\_ANNIHILATOR\_SEQ}(Z=Z, Z_c, E = E_{D_n^n}, S = S)^9;$ 
13: end if

```

5 Experiments and applications to W(A)PB functions

In this section, we apply the algorithms to the weightwise perfectly balanced (WPB) Boolean functions with the intent to study their algebraic immunity restricted to subsets of constant Hamming weight. In particular, we determine the probability distributions of the Al_k of WPB functions and we compute the Al_k of some WPB Boolean functions already studied. In addition, in Appendix A, we compare the execution time of Algorithm 3 and Algorithm 2. The source code implementing Algorithm 3 and Algorithm 2, along with

⁴ APPLY_OPERATIONS : applies column ops from P_{k-1} (Remark 8) to a vector.

⁵ EF_LAST_COL : variant of EF, echelonizing only last added row/col, not full matrix.

⁶ VANISHED_ON : given ANF coeffs of $g \in \mathcal{B}_n$ and point z , returns *True* if $g(z) = 0$, else *False*.

⁷ See Remark 10.

⁸ $\text{CONSTRUCT_MONOMIALS}$: generates all monomials up to degree d in n vars. $E_{D_n^n}$ can be pre-computed to skip this step.

⁹ $\text{FIND_DEG_SMALLEST_S_ANNIHILATOR_SEQ}$: like Algorithm 1 but finds degree sequentially, not in parallel.

the probability distributions and the values of Al_k for the published functions, is available in the public repository.

5.1 Determination of Al_k distribution of WPB functions

Similar to the approach in [25] for analyzing the weightwise nonlinearity NL_k , we determine the discrete probability distribution of Al_k for weightwise perfectly balanced Boolean functions. This distribution describes the probability that a function $f \in \mathcal{WPB}_m$ has a specific algebraic immunity restricted to the slice $E_{k,n}$. A formal definition is provided in Definition 16:

Definition 16. Let $m \in \mathbb{N}^*$, $n = 2^m$ and $k \in [1, n - 1]$, we define the discrete probability distribution of Al_k as:

$$p_{\text{Al}_k}(x) = \left| \{f \in \mathcal{WPB}_m \mid \text{Al}_k(f) = x\} \right| \cdot |\mathcal{WPB}_m|^{-1}$$

We aim to determine this distribution for $m = 2, 3$, and 4. However, the number of WPB functions increases fast with the growth of m , making this computation infeasible for $m > 2$. In fact, as stated in [25], $|\mathcal{WPB}_2| = 720$, while $|\mathcal{WPB}_3| > 2^{243}$ and $|\mathcal{WPB}_4| > 2^{65452}$. Therefore, we have to take a different approach for $m > 2$ by generating a certain number of samples from the set $\mathcal{WPB}_m$ and calculating the probability over it. The larger the sample size we select, the better we approximate the distribution. Table 1 presents the estimated probability distribution obtained.

Definition 17. We define the Al_k distribution over a sample Ω as:

$$p_{\Omega, \text{Al}_k}(x) = \left| \{f \in \Omega \mid \text{Al}_k(f) = x\} \right| \cdot |\Omega|^{-1}.$$

Determination of the full distribution for $m=2$ For $m = 2 \Rightarrow n = 4$, it is possible to determine the full probability distribution from Definition 16 without using the random sampling strategy. The resulting probability distribution table is presented in Table 1a.

Random Sampling In practice, random sampling is performed by generating a set $\Omega_k = \{\omega_1^{(k)}, \dots, \omega_{|\Omega|}^{(k)}\}$ consisting of $|\Omega|$ random balanced sub-truth tables for each slice $E_{k,n}$. Then, for each slice $E_{k,n}$, the Al_k is computed for each restricted set in Ω_k the frequency of each particular Al_k value is recorded. The following procedure is done for each slice $E_{k,n}$:

1. Initialize a hash table of all possible values of Al_k .
2. For each sub-truth table in $\omega_i^{(k)} \in \Omega_k$ (note that $\omega_i^{(k)}$ may be generated on the fly by an iterator).¹⁰:
 - (a) Run Algorithm 2 with S such that $f_S = \omega_i^{(k)}$.
 - (b) Increment the value in the hash table from the output of the previous step.
3. Divide each value in the hash table by $|\Omega|$.

Results for $n = 8$ and sample size = 2^{15} . Table 1b represents the Al_k distribution. We observe that for $n = 4$ all WPB functions have the same values of Al_k , for every k . However, for $n = 8$, there are WPB functions with different restricted Al_k , for example with $k = 4$ we observe functions reaching the lowest value (2, such as families exhibited in [25]) and some reaching 3 which is optimal for this size.

Results for $n = 16$ and sample size = 2^{10} . Table 1c represents the Al_k distribution. We observe that there are WPB functions with different restricted Al_k for this value of n , and that for the central values of k , all functions we sampled reach the optimal value of Al_k .

Table 1: Al_k probability distribution of $\mathcal{WPB}_2$, $\mathcal{WPB}_3$ and $\mathcal{WPB}_4$.

(a) Probability distribution p_{Al_k} for $n = 4, m = 2$.

$p(x), k$	0	1	2	3	4
0	1.00	0.00	0.00	0.00	1.00
1	0.00	1.00	0.00	1.00	0.00
2	0.00	0.00	1.00	0.00	0.00

(b) Probability distribution p_{Ω, Al_k} for $n = 8, m = 3, \Omega = 2^{15}$.

$p(x), k$	0	1	2	3	4	5	6	7	8
0	1.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.00
1	0.00	1.00	0.00	0.00	0.00	0.00	0.05	1.00	0.00
2	0.00	0.00	1.00	0.98	0.00	0.98	0.95	0.00	0.00
3	0.00	0.00	0.00	0.02	0.13	0.02	0.00	0.00	0.00
4	0.00	0.00	0.00	0.00	0.87	0.00	0.00	0.00	0.00

(c) Probability distribution p_{Ω, Al_k} for $n = 16, m = 4, \Omega = 2^{10}$.

$p(x), k$	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	1.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.00
1	0.00	1.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.00	0.00
2	0.00	0.00	1.00	0.02	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.01	1.00	0.00	0.00
3	0.00	0.00	0.00	0.98	0.13	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.13	0.99	0.00	0.00	0.00
4	0.00	0.00	0.00	0.00	0.87	0.63	0.00	0.00	0.00	0.00	0.00	0.60	0.87	0.00	0.00	0.00	0.00
5	0.00	0.00	0.00	0.00	0.00	0.37	1.00	0.00	0.00	0.00	1.00	0.40	0.00	0.00	0.00	0.00	0.00
6	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.00	1.00	1.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00

5.2 Determination of Al_k of known WPB families

In this part, we determine the Al_k s of functions introduced in [63], [18], [57] and in [7] for $n = 8$ and 16 variables.

The results are summarized in Table 2. Of the four WPB calculations we performed, the one in [57] is the one that would resist the best to algebraic attacks (with constant Hamming weight inputs), as it achieves higher Al_k values for both $n = 8$ and $n = 16$. In contrast, the one in [63], would be the least resistant, as it has the overall lower Al_k for both $n = 8$ and $n = 16$. Specifically:

- $n = 8$. The function in [63] has a constant Al_k of 2 for $k \in [2, 6]$ and the functions in [7] a constant Al_k of 2 for $k \in [3, 6]$, whereas the functions in [18] and in [57] achieve an Al_4 of 3.
- $n = 16$. The function in [63] again has a constant Al_k of 2 for $k \in [2, 14]$. In contrast, the function in [57] achieves an Al_k of 3 for $k = 4, 5, 11$ and 12, an Al_k of 4 for $k = 6, 7, 9$ and 10 and an Al_k of

¹⁰ Step 2 may be executed in parallel on the $\omega_i^{(k)}$ s if memory is not a constraint. In fact, depending on the value of m and the chosen sample size, the number of sub-truth tables generated may be too large to fit in memory simultaneously.

5 for $k = 8$. The functions [18] and in [7], both reach an Al_k of 3 but for $k \in [4, 12]$ and $k \in [5, 13]$ respectively.

The maximum execution times for [18], [63], [57] and [7] with $n = 8$ occur with $k = 4$ and are around 0.014, 0.029, 0.011 and 0.012 seconds, respectively. For $n = 16$, the maximum execution times for [63], [57] and [7] occur with $k = 8$ and are 5.898671, 126.627561 and 12.048051 seconds respectively. In contrast, for [18], the maximum execution time occurs for $k = 9$ and is 7.935819 seconds.

Table 2: Al_k of the construction from [18], from [63], from [57] and from [7] with $m = 3$ ($n = 8$) and $m = 4$ ($n = 16$).

[18]																	
n, k	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
8	0	1	1	2	3	2	2	1	0								
16	0	1	1	2	3	3	3	3	3	3	3	3	2	2	1	0	
[63]																	
n, k	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
8	0	1	2	2	2	2	2	1	0								
16	0	1	2	2	2	2	2	2	2	2	2	2	2	2	2	1	0
[57]																	
n, k	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
8	0	1	2	2	3	2	2	1	0								
16	0	1	2	2	3	3	4	4	5	4	4	3	3	2	2	1	0
[7]																	
n, k	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
8	0	1	1	2	2	2	2	1	0								
16	0	1	1	2	2	3	3	3	3	3	3	3	3	3	2	1	0

In Table 3, we summarize the results on Al_k for WPB functions with 8 variables, including both our findings and those from the state of the art. Similarly, Table 4 presents the corresponding results for functions with 16 variables. We observe that most of the studied families exhibit lower weightwise algebraic immunity than a randomly chosen WPB function, with the exception of the field order construction [45] (usually a function is considered good if its parameters are the same or better than the ones of a random function).

Function	Al_2	Al_3	Al_4	Al_5	Al_6
[18]	1	2	3	2	2
[63]	2	2	2	2	2
[57]	2	2	3	2	2
[7]	1	2	2	2	2
Estimated average	1.95	2.02	2.88	2.02	1.95
Field order [45]	1	2	3	2	2
Upper bound	2	3	3	3	2

Table 3: Al_k of WPB functions in 8 variables.

Function	Al_2	Al_3	Al_4	Al_5	Al_6	Al_7	Al_8	Al_9	Al_{10}	Al_{11}	Al_{12}	Al_{13}	Al_{14}
[18]	1	2	3	3	3	3	3	3	3	3	3	2	2
[63]	2	2	2	2	2	2	2	2	2	2	2	2	2
[57]	2	2	3	3	4	4	5	4	4	3	3	2	2
[7]	1	2	2	3	3	3	3	3	3	3	3	3	2
Estimated average	2.00	2.99	3.89	4.37	5.00	6.00	6.00	6.00	5.00	4.40	3.87	2.99	2.00
Field order [45]	2	3	4	4	5	6	6	6	5	4	3	2	2
Upper bound	2	3	4	5	5	6	6	6	5	5	4	3	2

Table 4: Al_k of WPB functions in 16 variables.

6 Conclusion

In this article, we formalized the main theoretical foundations and provided algorithms for computing restricted algebraic immunity over any set $S \subseteq \mathbb{F}_2^n$, using both the Reed-Muller and iterative approaches. We compared the practical complexity of these algorithms and showed that the iterative approach is more efficient. Additionally, we applied our methods to compute or estimate, for the first time, the distribution of Al_k for WPB functions with 4, 8, and 16 variables. Furthermore, we determined the values of Al_k for various families of WPB functions introduced in previous works and showed that, in most cases, these functions exhibit lower weightwise algebraic immunity than a randomly chosen WPB function.

By formalizing the theoretical background and providing both algorithms and implementations for computing restricted algebraic immunity, we hope this work will contribute to further research and the practical implementation of algebraic attack variants.

7 Acknowledgments

Pierrick Méaux was supported by the ERC Advanced Grant no. 787390.

This work was conducted in the context of Luca Bonamino's M.Sc. thesis in the Master of Information and Computer Science (MICS) programme run by the Department of Computer Science (DCS) of the University of Luxembourg.

References

1. Applebaum, B., Lovett, S.: Algebraic attacks against random local functions and their countermeasures. *SIAM J. Comput.* pp. 52–79 (2018)
2. Armknecht, F.: Improving fast algebraic attacks. In: *Fast Software Encryption 2004. Lecture Notes in Computer Science*, vol. 3017, pp. 65–82. Springer (2004)
3. Armknecht, F., Carlet, C., Gaborit, P., Künzli, S., Meier, W., Ruatta, O.: Efficient computation of algebraic immunity for algebraic and fast algebraic attacks. In: Vaudenay, S. (ed.) *EUROCRYPT 2006. Lecture Notes in Computer Science*, vol. 4004, pp. 147–164. Springer (2006)
4. Carlet, C.: Constructing balanced functions with optimum algebraic immunity. In: *IEEE International Symposium on Information Theory, ISIT 2007, Nice, France, June 24–29, 2007*. pp. 451–455. IEEE (2007). <https://doi.org/10.1109/ISIT.2007.4557094>
5. Carlet, C.: *Boolean Functions for Cryptography and Coding Theory*. Cambridge University Press (2021). <https://doi.org/10.1017/9781108606806>
6. Carlet, C., Feng, K.: An infinite class of balanced functions with optimal algebraic immunity, good immunity to fast algebraic attacks and good nonlinearity. In: *Advances in Cryptology - ASIACRYPT 2008*. pp. 425–440 (2008)
7. Carlet, C., Méaux, P., Rotella, Y.: Boolean functions with restricted input and their robustness; application to the FLIP cipher. *IACR Trans. Symmetric Cryptol.* **2017**(3) (2017)
8. Carlet, C., Merabet, B.: Asymptotic lower bound on the algebraic immunity of random balanced multi-output boolean functions. *Adv. Math. Commun.* **7**(2), 197–217 (2013). <https://doi.org/10.3934/AMC.2013.7.197>
9. Carlet, C., Sarkar, P.: Constructions of efficiently implementable boolean functions possessing high nonlinearity and good resistance to algebraic attacks. *IACR Cryptol. ePrint Arch.* p. 1305 (2024), <https://eprint.iacr.org/archive/2024/1305/20240821:123900.5>
10. Carlet, C., Sarkar, P.: The nonlinear filter model of stream cipher redivivus. *Cryptology ePrint Archive*, Paper 2025/160 (2025), <https://eprint.iacr.org/2025/160>
11. Chacal, N., Méaux, P., AnonymousBeforeAcceptance: Nostalgia cipher: can filtered lfsrs be secure again? *ORBilu Archive* (2025), <https://hdl.handle.net/10993/64454>
12. Chen, Y., Lu, P.: Two classes of symmetric boolean functions with optimum algebraic immunity: Construction and analysis. *IEEE Transactions on Information Theory* **57**(4), 2522–2538 (April 2011). <https://doi.org/10.1109/TIT.2011.2111810>
13. Cosserson, O., Hoffmann, C., Méaux, P., Standaert, F.: Towards case-optimized hybrid homomorphic encryption - featuring the Elisabeth stream cipher. In: Agrawal, S., Lin, D. (eds.) *ASIACRYPT 2022, Part III. LNCS*, vol. 13793, pp. 32–67. Springer (2022). https://doi.org/10.1007/978-3-031-22969-5_2
14. Courtois, N.T.: Fast algebraic attacks on stream ciphers with linear feedback. In: Boneh, D. (ed.) *Advances in Cryptology - CRYPTO 2003. Lecture Notes in Computer Science*, vol. 2729, pp. 176–194. Springer (2003)
15. Courtois, N.T., Meier, W.: Algebraic attacks on stream ciphers with linear feedback. In: Biham, E. (ed.) *Advances in Cryptology - EUROCRYPT 2003. Lecture Notes in Computer Science*, vol. 2656, pp. 345–359. Springer (2003)
16. Dalai, D.K., Gupta, K.C., Maitra, S.: Results on algebraic immunity for cryptographically significant boolean functions. In: *Progress in Cryptology - INDOCRYPT 2004, 5th International Conference on Cryptology in India, Chennai, India, December 20–22, 2004, Proceedings. Lecture Notes in Computer Science*, vol. 3348, pp. 92–106. Springer (2004). https://doi.org/10.1007/978-3-540-30556-9_9
17. Dalai, D.K., Maitra, S., Sarkar, S.: Basic theory in construction of boolean functions with maximum possible annihilator immunity. *Designs, Codes and Cryptography* (2006)
18. Dalai, D.K., Mallick, K.: A class of weightwise almost perfectly balanced boolean functions. *Advances in Mathematics of Communications* **18**(2), 480–504 (2024)
19. Dalai, D.K., Mallick, K., Méaux, P.: Weightwise almost perfectly balanced functions, construction from A permutation group action view. *IACR Cryptol. ePrint Arch.* p. 2068 (2024), <https://eprint.iacr.org/2024/2068>
20. Didier, F.: A new upper bound on the block error probability after decoding over the erasure channel. *IEEE Trans. Inf. Theory* **52**(10), 4496–4503 (2006)

21. Didier, F., Tillich, J.: Computing the algebraic immunity efficiently. In: Robshaw, M.J.B. (ed.) Fast Software Encryption 2006. Lecture Notes in Computer Science, vol. 4047, pp. 359–374. Springer (2006)
22. Gilbert, H., Boissier, R.H., Jean, J., Reinhard, J.: Cryptanalysis of elisabeth-4. In: Guo, J., Steinfeld, R. (eds.) Advances in Cryptology - ASIACRYPT 2023 - 29th International Conference on the Theory and Application of Cryptology and Information Security, Guangzhou, China, December 4–8, 2023, Proceedings, Part III. Lecture Notes in Computer Science, vol. 14440, pp. 256–284. Springer (2023). https://doi.org/10.1007/978-981-99-8727-6_9
23. Gini, A., Méaux, P.: On the weightwise nonlinearity of weightwise perfectly balanced functions. Discret. Appl. Math. **322**, 320–341 (2022). <https://doi.org/10.1016/j.dam.2022.08.017>
24. Gini, A., Méaux, P.: Weightwise almost perfectly balanced functions: Secondary constructions for all n and better weightwise nonlinearities. In: Isobe, T., Sarkar, S. (eds.) Progress in Cryptology - INDOCRYPT 2022. Lecture Notes in Computer Science, vol. 13774, pp. 492–514. Springer (2022)
25. Gini, A., Méaux, P.: On the algebraic immunity of weightwise perfectly balanced functions. In: Aly, A., Tibouchi, M. (eds.) Progress in Cryptology - LATINCRYPT 2023. Lecture Notes in Computer Science, vol. 14168, pp. 3–23. Springer (2023). https://doi.org/10.1007/978-3-031-44469-2_1
26. Gini, A., Méaux, P.: Weightwise perfectly balanced functions and nonlinearity. In: Hajji, S.E., Mesnager, S., Souidi, E.M. (eds.) Codes, Cryptology and Information Security 2023. Lecture Notes in Computer Science, vol. 13874, pp. 338–359. Springer (2023). https://doi.org/10.1007/978-3-031-33017-9_21
27. Gini, A., Méaux, P.: S_0 -equivalence classes, a new direction to find better weightwise perfectly balanced functions, and more. Cryptogr. Commun. **16**(6), 1211–1234 (2024). <https://doi.org/10.1007/S12095-024-00719-W>
28. Goldreich, O.: Candidate one-way functions based on expander graphs. Electronic Colloquium on Computational Complexity (ECCC) **7**(90) (2000)
29. Guo, X., Su, S.: Construction of weightwise almost perfectly balanced boolean functions on an arbitrary number of variables. Discrete Applied Mathematics **307**, 102–114 (2022)
30. Hawkes, P., Rose, G.G.: Rewriting variables: The complexity of fast algebraic attacks on stream ciphers. In: Franklin, M.K. (ed.) Advances in Cryptology - CRYPTO 2004. Lecture Notes in Computer Science, vol. 3152, pp. 390–406. Springer (2004)
31. Hoffmann, C., Méaux, P., Standaert, F.: The patching landscape of elisabeth-4 and the mixed filter permutator paradigm. In: Chattopadhyay, A., Bhasin, S., Picek, S., Rebeiro, C. (eds.) INDOCRYPT 2023, Part I. LNCS, vol. 14459, pp. 134–156. Springer (2023). https://doi.org/10.1007/978-3-031-56232-7_7
32. Lauter, K., Naehrig, M., Vaikuntanathan, V.: Can homomorphic encryption be practical? Cryptology ePrint Archive, Report 2011/405 (2011)
33. Li, J., Carlet, C., Zeng, X., Li, J., Hu, L., Shan, J.: Two constructions of balanced boolean functions with optimal algebraic immunity, high nonlinearity and good behavior against fast algebraic attacks. Designs, Codes and Cryptography **76** (03 2014)
34. Li, J., Su, S.: Construction of weightwise perfectly balanced boolean functions with high weightwise nonlinearity. Discret. Appl. Math. **279**, 218–227 (2020)
35. Li, N., Qi, W.: Boolean functions of an odd number of variables with maximum algebraic immunity. Science in China Series F: Information Sciences pp. 307–317 (2007), <https://doi.org/10.1007/s11432-007-0027-4>
36. Limnietis, K., Kolokotronis, N.: Boolean functions with maximum algebraic immunity: further extensions of the carlet–feng construction. DCC **86**, 1685–1706 (2018)
37. Limnietis, K., Kolokotronis, N., Kalouptsidis, N.: Secondary constructions of boolean functions with maximum algebraic immunity. Cryptography and Communications **5** (09 2013)
38. Liu, J., Mesnager, S.: Weightwise perfectly balanced functions with high weightwise nonlinearity profile. Des. Codes Cryptogr. **87**(8), 1797–1813 (2019)
39. Liu, M., Lin, D.: Almost perfect algebraic immune functions with good nonlinearity. In: 2014 IEEE International Symposium on Information Theory. pp. 1837–1841 (June 2014). <https://doi.org/10.1109/ISIT.2014.6875151>
40. Liu, M., Zhang, Y., Lin, D.: Perfect algebraic immune functions. In: Wang, X., Sako, K. (eds.) Advances in Cryptology - ASIACRYPT 2012 - 18th International Conference on the Theory and Application of Cryptology and Information Security, Beijing, China, December 2–6, 2012. Proceedings. Lecture Notes in Computer Science, vol. 7658, pp. 172–189. Springer (2012). https://doi.org/10.1007/978-3-642-34961-4_12
41. Mandujano, S., Ku Cauich, J.C., Lara, A.: Studying special operators for the application of evolutionary algorithms in the seek of optimal boolean functions for cryptography. In: Advances in Computational Intelligence. pp. 383–396. Springer Nature Switzerland (2022)
42. Mariot, L., Picek, S., Jakobovic, D., Djurasevic, M., Leporati, A.: Evolutionary construction of perfectly balanced boolean functions. In: 2022 IEEE Congress on Evolutionary Computation (CEC). p. 1–8. IEEE Press (2022)
43. Méaux, P.: On the algebraic immunity of direct sum constructions. Discret. Appl. Math. **320**, 223–234 (2022). <https://doi.org/10.1016/j.dam.2022.05.021>
44. Méaux, P.: From at least $n/3$ to at most $3\sqrt{n}$: Correcting the algebraic immunity of the hidden weight bit function. IACR Cryptol. ePrint Arch. p. 615 (2025), <https://eprint.iacr.org/2025/615>, to appear at Latincrypt 2025

45. Méaux, P.: Weightwise (almost) perfectly balanced functions based on total orders. In: Eichlseder, M., Gambs, S. (eds.) *Selected Areas in Cryptography – SAC 2024*. pp. 112–138. Springer Nature Switzerland, Cham (2025)
46. Méaux, P., Carlet, C., Journault, A., Standaert, F.: Improved filter permutators for efficient FHE: better instances and implementations. In: *Progress in Cryptology - INDOCRYPT. LNCS*, vol. 11898, pp. 68–91. Springer (2019)
47. Méaux, P., Journault, A., Standaert, F.X., Carlet, C.: Towards stream ciphers for efficient FHE with low-noise ciphertexts. In: *EUROCRYPT*. pp. 311–343 (2016)
48. Méaux, P., Seuré, T., Tang, D.: The revisited hidden weight bit function. *IACR Cryptol. ePrint Arch.* p. 2022 (2024), <https://eprint.iacr.org/2024/2022>, to appear at *Selected Areas in Cryptography – SAC 2025*
49. Méaux, P., Wang, Q.: Towards a generalization of the algebraic attack on stream ciphers: A study of the case with only extremal-degree monomials. *IACR Commun. Cryptol.* **2**(1), 29 (2025). <https://doi.org/10.62056/ABY7QJP10>
50. Meier, W., Pasalic, E., Carlet, C.: Algebraic attacks and decomposition of boolean functions. In: Cachin, C., Camenisch, J.L. (eds.) *Advances in Cryptology - EUROCRYPT 2004*. pp. 474–491. Springer Berlin Heidelberg, Berlin, Heidelberg (2004)
51. Mesnager, S., Su, S.: On constructions of weightwise perfectly balanced boolean functions. *Cryptography and Communications* (2021)
52. Mesnager, S., Su, S., Li, J., Zhu, L.: Concrete constructions of weightwise perfectly balanced (2-rotation symmetric) functions with optimal algebraic immunity and high weightwise nonlinearity. *Cryptogr. Commun.* **14**(6), 1371–1389 (2022)
53. Qu, L., Feng, K., Liu, F., Wang, L.: Constructing symmetric boolean functions with maximum algebraic immunity. *IEEE Transactions on Information Theory* **55**, 2406–2412 (05 2009). <https://doi.org/10.1109/TIT.2009.2015999>
54. Stevens, B., Williams, A.: The coolest order of binary strings. In: Kranakis, E., Krizanc, D., Luccio, F.L. (eds.) *Fun with Algorithms 2012. Lecture Notes in Computer Science*, vol. 7288, pp. 322–333. Springer (2012)
55. Tang, D., Carlet, C., Tang, X., Zhou, Z.: Construction of highly nonlinear 1-resilient boolean functions with optimal algebraic immunity and provably high fast algebraic immunity. *IEEE Transactions on Information Theory* pp. 6113–6125 (Sep 2017). <https://doi.org/10.1109/TIT.2017.2725918>
56. Tang, D., Carlet, C., Tang, X.: Highly nonlinear boolean functions with optimal algebraic immunity and good behavior against fast algebraic attacks. *IEEE Transactions on Information Theory* pp. 653–664 (01 2013). <https://doi.org/10.1109/TIT.2012.2217476>
57. Tang, D., Liu, J.: A family of weightwise (almost) perfectly balanced boolean functions with optimal algebraic immunity. *Cryptogr. Commun.* **11**(6), 1185–1197 (2019)
58. Tang, D., Luo, R., Du, X.: The exact fast algebraic immunity of two subclasses of the majority function. *IEICE Transactions* pp. 2084–2088 (2016)
59. Tu, Z., Deng, Y.: A conjecture about binary strings and its applications on constructing boolean functions with optimal algebraic immunity. *DCC* **60**, 1–14 (2011)
60. Yan, L., Cui, J., Liu, J., Xu, G., Han, L., Jolfaei, A., Zheng, X.: Iga: An improved genetic algorithm to construct weightwise (almost) perfectly balanced boolean functions with high weightwise nonlinearity. In: *ASIA CCS 2023*. p. 638–648 (2023)
61. Zeng, X., Carlet, C., Shan, J., Hu, L.: More balanced boolean functions with optimal algebraic immunity and good nonlinearity and resistance to fast algebraic attacks. *IEEE Transactions on Information Theory* **57**(9), 6310–6320 (2011)
62. Zhang, R., Su, S.: A new construction of weightwise perfectly balanced boolean functions. *Adv. Math. Commun.* **17**(4), 757–770 (2023). <https://doi.org/10.3934/AMC.2021020>
63. Zhao, Q., Jia, Y., Zheng, D., Qin, B.: A new construction of weightwise perfectly balanced functions with high weightwise nonlinearity. *Mathematics* **11**(5) (2023)
64. Zhao, Q., Li, M., Chen, Z., Qin, B., Zheng, D.: A unified construction of weightwise perfectly balanced boolean functions. *Discrete Applied Mathematics* **337**, 190–201 (2023)
65. Zhu, L., Su, S.: A systematic method of constructing weightwise almost perfectly balanced boolean functions on an arbitrary number of variables. *Discrete Applied Mathematics* **314**, 181–190 (2022)

A Comparison between Algorithm 2 and Algorithm 3 on WAPB functions

In this part, we compare the execution times of Algorithm 3 and Algorithm 2 by running both algorithms on multiple WAPB functions restricted to the largest set $E_{\lceil n/2 \rceil, n}$, with different numbers of variables. Specifically, the experiment is done by running the algorithms with the number of variables n going from 1 to 12. For both algorithms, if $n \leq 4$, we calculate the average over the $Al_{\lceil n/2 \rceil}$ of all functions in $\mathcal{WAPB}_n$, whereas for $n > 4$, we calculate the average over the $Al_{\lceil n/2 \rceil}$ of a random sample of 10^4 $\mathcal{WAPB}_n$ functions.

Figure 1 shows the mean execution time per value of n of the two algorithms. We denote by $T(G^{\leq n_{max}}) = T(G^{\leq 12})$ the necessary time to pre-compute all the necessary Reed-Muller codes's generator matrices following Equation (2): we recall that it is the necessary time to compute the sequence $(G_{n,n})_{n \in [1, 12=n_{max}]}$, not only the generator matrix $G_{n,n}$ for the current iteration n . In fact, $T(G^{\leq 12})$ is constant, that is because the computation occurs before beginning the experiment since it is expected that Algorithm 3 is able to run for any number of variable n up to an upper bound n_{max} , without having to compute the Reed-Muller code for it.

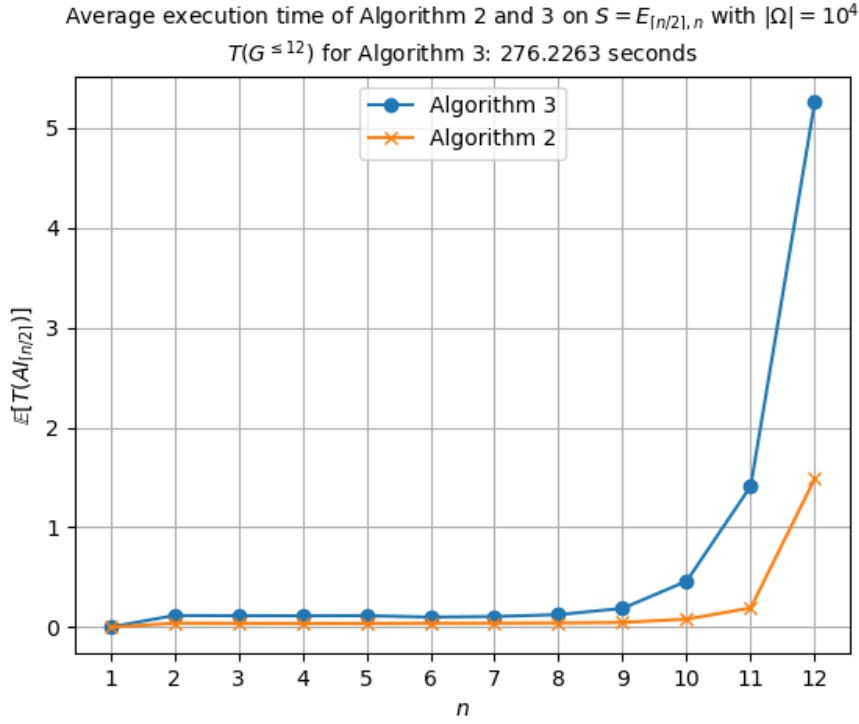


Fig. 1: Average time execution per number of variables of Algorithms 3 and 2 over 10^4 random $\mathcal{WAPB}_n$ functions on the set $E_{\lceil n/2 \rceil, n}$. $T(G^{\leq 12})$ gives the time of the pre-computation of the sequence of all Reed-Muller matrices up to $G_{12,12}$.

The pre-computation times of $G^{\leq n_{max}}$ for Algorithm 3 are given in the Table 5. From Figure 1, we observe that Algorithm 2 outperforms Algorithm 3 for $n \leq 12$, even without considering the pre-computation time required for $G^{\leq n_{max}}$ and $D^{\leq n_{max}}$ in Algorithm 3. Furthermore, Algorithm 3 becomes

impractical for $n > 12$ due to the excessive complexity of pre-computation. In contrast, Algorithm 2 remains feasible for larger values of n .

$G^{\leq 1}$	$G^{\leq 2}$	$G^{\leq 3}$	$G^{\leq 4}$	$G^{\leq 5}$	$G^{\leq 6}$	$G^{\leq 7}$	$G^{\leq 8}$	$G^{\leq 9}$	$G^{\leq 10}$	$G^{\leq 11}$	$G^{\leq 12}$
0.0126	0.0042	0.0017	0.004	0.0149	0.0470	0.1938	0.806	3.5456	14.9168	65.5014	276.2263

Table 5: Pre-computation time (in seconds) for $G^{\leq n_{max}}$ for n_{max} from 1 to 12.

Both implementations of the algorithms used to generate the plot in Figure 1 have been developed in *Python*, with the following enhancements:

- The linear algebra operations in Algorithm 3 are performed using the *SageMath* library, while those in Algorithm 2 are executed using an ad-hoc custom *Python* package implemented in *Rust*. This setup ensures comparability between the two implementations, as the *SageMath* library is highly optimized for linear algebra operations. However, for the results reported in Section 5, a full implementation in *Rust* of Algorithm 2 has been used.
- The computation of $G^{\leq n_{max}}$ for Algorithm 3 is performed using the *generator_matrix* method of the *SageMath* object *BinaryReedMullerCodes*. While this computation could be more efficient if implemented in *Rust*, the current overall implementation still faces scalability issues. Specifically, for $n \geq 16$, the algorithm becomes impractical because $G_{16,16}$ cannot be stored in a native *Python* list or a *SageMath* matrix object. However, if the entire algorithm were implemented in *Rust*, it could potentially be feasible. Consequently, for larger values of n , Algorithm 3 is not usable, while Algorithm 2 remains viable.

B Algorithms

Algorithm 3 Algebraic immunity of f restricted to the set S , Reed-Muller method.

Input: $x \in \mathbb{F}_2^{2^n}$ evaluation of the truth table of $f \in \mathcal{B}_n$ and the restricted set $S \subseteq \mathbb{F}_2^n$.

Output: $\text{Al}_S(f)$

```

1: function FIND_DEG_SMALLEST_S_ANNIHILATOR( $G_{n,n}^{f_Z^{(o)},S}, G_{d,n}^S, D^n$ )
2:    $r, i, \text{previousIndex} \leftarrow 0, 0, 0$ ;
3:    $ef_f, ef_S \leftarrow \text{null}, \text{null}$ ;
4:   while  $r \leq |S|/2$  do
5:      $D_i^n \leftarrow D^n[i]$ ;
6:      $ef_f \leftarrow \text{ECHELON\_FORM\_LAST\_}D_i^n\text{-ROWS} \left( ef_f \parallel G_{n,n}^{f_Z^{(o)},S}[\text{previousIndex} : D_i^n] \right)^{11}$ ;
7:      $ef_S \leftarrow \text{ECHELON\_FORM\_LAST\_}D_i^n\text{-ROWS} (ef_S \parallel G_{n,n}^S[\text{previousIndex} : D_i^n])$ ;
8:      $r \leftarrow \text{rank}(ef_S)$ ;
9:     if  $\text{rank}(ef_f) < r$  then
10:      return  $i$ ;
11:     end if
12:      $\text{previousIndex} \leftarrow D_i^n$ ;
13:      $i \leftarrow i + 1$ ;
14:   end while
15: end function
16:
17:  $Z, Z_c \leftarrow \text{supp}(x) \cap S, \text{supp}(x+1) \cap S$ ;
18: if  $Z = \emptyset$  of  $Z_c = \emptyset$  then return 0;
19: end if
20:  $m_{max}, D^{\leq n_{max}} \leftarrow \text{READ\_VALUES\_FROM\_FILE}()$ ;
21:  $G_{n,n} \leftarrow m_{max}[n]$ ;
22:  $D^n \leftarrow D^{\leq n_{max}}[n]$ ;
23:  $G_{n,n}^{f_Z,S}, G_{n,n}^{(f+1)Z,S}, G_{n,n}^S \leftarrow \text{CONSTRUCT\_RESTRICTED\_GENERATOR\_MATRICES}(G_{n,n}, x, S)^{12}$ 
24:  $\text{immunity}_f, \text{immunity}_{f+1} \leftarrow ($ 
25:   FIND_DEG_SMALLEST_S_ANNIHILATOR( $G_{n,n}^{f_Z,S}, G_{n,n}^S, D^n$ )
26:   | FIND_DEG_SMALLEST_S_ANNIHILATOR( $G_{n,n}^{(f+1)Z,S}, G_{n,n}^S, D^n$ )
27: );
28: return  $\text{MIN}(\text{immunity}_f, \text{immunity}_{f+1})^{13}$ ;
```

¹¹ The symbol $\parallel$ is used as row-concatenation of matrices, meaning that $C = A \parallel B$ is the matrix constructed by appending all the row of the matrix B to the matrix A .

¹² CONSTRUCT_RESTRICTED_GENERATOR_MATRICES is a function extracting $G_{n,n}^{f_{\text{supp}(f) \cap S}, S}$, $G_{n,n}^{f_{\text{supp}(f+1) \cap S}, S}$ and $G_{n,n}^S$ from the full Reed-Muller code generator matrix $G_{n,n}$ following Definition 11.

¹³ The function MIN takes care of *null* values: $\text{MIN}(\text{immunity}_f, \text{null}) = \text{immunity}_f$ and $\text{MIN}(\text{null}, \text{immunity}_{f+1}) = \text{immunity}_{f+1}$.